

Docker 入门学习笔记

镜像 · 容器 · 端口 · 挂载 · 网络 · Compose · Dockerfile

```
$ docker compose up -d  
$ docker logs -f app  
$ docker compose down
```

Docker 入门学习笔记

根据 Docker 视频课程整理：镜像、容器、端口、挂载、网络、Compose、Dockerfile 与排障

最近看了一套 Docker 视频课，目录很清晰：先讲 Docker 是什么，再讲镜像、容器、`docker run`、挂载、网络、Compose、Dockerfile，最后讲如何销毁和排查。看完以后我最大的感受是：Docker 不是一堆命令的堆砌，而是一套把应用环境固定下来的方法。

以前部署一个网站，常见步骤是：服务器装 Node，装 pnpm，拉代码，装依赖，配置环境变量，启动进程，再想办法守护进程。问题是，每台服务器都可能不一样。今天能跑，换一台机器就可能因为 Node 版本、系统依赖、端口、权限出各种问题。Docker 想解决的就是这个问题：把运行环境一起打包，让应用在不同机器上尽量表现一致。

这篇笔记按视频课程的顺序整理，也夹一点我自己的理解。

1. Docker 到底解决什么问题

Docker 的核心不是“虚拟机”，而是“容器化”。它把一个应用运行需要的东西放到一个隔离环境里，比如：

- 操作系统层面的基础环境
- Node、Python、Nginx、Redis 之类的运行时
- 项目依赖
- 启动命令

- 端口映射
- 文件挂载
- 网络关系

这样部署时就不用服务器上手动配一堆环境。你只要有 Docker，就可以启动容器。

视频里讲 Docker 架构时，可以把它理解成三层：

镜像 image：应用和环境的模板
容器 container：镜像运行起来后的实例
仓库 registry：存放和分发镜像的地方

这三个概念贯穿后面所有命令。

2. 镜像：应用环境的模板

镜像像一个只读模板。比如 `nginx:latest`、`redis:7`、`node:22-alpine` 都是镜像。

常用命令：

```
docker images
docker pull nginx
docker rmi nginx
```

`docker pull nginx` 的意思是从镜像仓库下载 nginx 镜像。下载完以后，本机就有了一个可以启动 nginx 容器的模板。

镜像本身不会运行，运行起来才叫容器。这个关系有点像：

类 -> 对象
镜像 -> 容器

同一个镜像可以启动多个容器，每个容器都有自己的进程、文件层和网络环境。

3. 容器：真正跑起来的服务

容器是镜像运行后的实例。比如：

```
docker run nginx
```

这会用 nginx 镜像启动一个容器。但这个命令不太适合长期服务，因为它会占住当前终端。更常见的写法是：

```
docker run -d --name my-nginx -p 8080:80 nginx
```

这条命令里有几个重点：

-d : 后台运行
--name my-nginx : 给容器取名
-p 8080:80 : 把宿主机 8080 端口映射到容器 80 端口
nginx : 使用 nginx 镜像

查看容器：

```
docker ps
docker ps -a
```

停止和删除：

```
docker stop my-nginx
docker rm my-nginx
```

这里要记住一个特别重要的点：删除容器不等于删除镜像。容器是运行实例，镜像是模板。

4. 端口映射：为什么是 8080:80

视频里讲 `docker run` 细节时，端口映射是非常关键的一块。

格式是：

宿主机端口:容器端口

例如：

```
docker run -d -p 8080:80 nginx
```

意思是：

访问服务器的 8080 端口
转发到容器里的 80 端口

所以浏览器访问：

http://服务器IP:8080

容器内部实际接收的是 80 端口。

很多人刚学 Docker 时会把左右写反。判断方法很简单：左边是你在服务器外面访问的端口，右边是应用在容器里面监听的端口。

5. 日志：排错第一入口

容器启动失败时，第一反应不要重装，不要乱删，先看日志：

```
docker logs 容器名
docker logs -f 容器名
```

如果用 Compose :

```
docker compose logs -f
docker compose logs -f 服务名
```

日志通常会告诉你 :

- 依赖下载失败
- 端口冲突
- 环境变量缺失
- 数据库连不上
- 权限不足
- 应用启动命令错误

这比盲目猜要靠谱得多。

6. 挂载 : 容器删了 , 数据怎么办

视频里讲存储时提到两种常见方式 :

目录挂载 bind mount
数据卷 volume

目录挂载例子 :

```
docker run -d \
-v /opt/nginx/html:/usr/share/nginx/html \
-p 8080:80 \
nginx
```

这里的意思是 :

宿主机 /opt/nginx/html
挂载到容器 /usr/share/nginx/html

容器里读写这个目录 , 本质上就是读写宿主机的目录。

数据卷例子 :

```
docker volume create mysql-data
docker run -d \
-v mysql-data:/var/lib/mysql \
mysql
```

目录挂载适合你想直接看到文件的场景 , 比如配置文件、上传文件。数据卷适合交给 Docker 管理 , 比如数据库数据。

一定要记住：容器不是数据保险箱。容器删掉后，如果数据没有挂载到宿主机或 volume，数据就可能一起没了。

7. 网络：容器之间怎么互相访问

如果只有一个容器，端口映射就够了。可一旦有多个容器，比如 Web 服务连 Redis、MySQL，就需要理解 Docker 网络。

创建网络：

```
docker network create app-net
```

启动 Redis：

```
docker run -d --name redis --network app-net redis
```

启动应用：

```
docker run -d --name app --network app-net my-app
```

在同一个 Docker 网络里，容器可以通过容器名访问彼此。例如应用里连接 Redis，可以写：

```
redis:6379
```

而不是写 `127.0.0.1:6379`。

这是一个非常容易踩坑的地方。容器里的 `127.0.0.1` 指的是容器自己，不是宿主机，也不是别的容器。

8. Docker Compose：把一堆 docker run 写成一个文件

当命令越来越长时，继续手写 `docker run` 就不舒服了。比如要写：

- 镜像名
- 容器名
- 端口
- 环境变量
- 挂载目录
- 网络
- 重启策略

Docker Compose 就是把这些配置写到 `docker-compose.yml` 里。

一个简单例子：

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    restart: unless-stopped
```

启动：

```
docker compose up -d
```

停止并删除：

```
docker compose down
```

查看状态：

```
docker compose ps
```

查看日志：

```
docker compose logs -f
```

视频里用 WordPress 演示 Compose 很典型，因为 WordPress 不是一个容器就完事，它还需要数据库。Compose 的价值就在这里：多个服务、多个端口、多个 volume、多个环境变量，可以统一写在一个文件里。

9. Compose 文件怎么看

一个稍微完整一点的 Compose 文件大概长这样：

```
services:
  app:
    image: my-app:latest
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
    volumes:
      - ./data:/app/data
```

逐项理解：

services：定义服务
app：服务名
image：使用哪个镜像
restart：重启策略
ports：端口映射
environment：环境变量
volumes：目录或数据卷挂载

`restart: unless-stopped` 很常用。意思是容器异常退出会自动重启，除非你手动停止它。

10. Dockerfile：自己制作镜像

如果 Docker Compose 是“怎么运行服务”，那 Dockerfile 就是“怎么制作镜像”。

一个最简单的 Node 项目 Dockerfile：

```
FROM node:22-alpine

WORKDIR /app

COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile

COPY . .
RUN pnpm build

EXPOSE 3000
CMD ["pnpm", "start"]
```

常见指令：

```
FROM：基于哪个基础镜像
WORKDIR：设置工作目录
COPY：复制文件
RUN：构建阶段执行命令
EXPOSE：声明容器端口
CMD：容器启动命令
```

这里要分清两个命令：

```
RUN：构建镜像时执行
CMD：容器启动时执行
```

比如安装依赖、编译项目，应该放在 `RUN`。启动服务，应该放在 `CMD`。

11. 镜像分层：为什么 Dockerfile 顺序重要

视频里讲镜像分层机制时，有一个实践非常重要：变化少的东西放前面，变化多的东西放后面。

比如 Node 项目通常这样写：

```
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile

COPY . .
RUN pnpm build
```

为什么不直接 `COPY . .` 再安装依赖？

因为 Docker 会缓存每一层。如果只改了业务代码，没有改 `package.json` 和锁文件，依赖安装这一层就可以复用缓存，构建会快很多。

这也是为什么 Dockerfile 不只是“能跑就行”，写得好不好会直接影响构建速度和维护体验。

12. 多阶段构建：让最终镜像更干净

实际项目里，经常会用多阶段构建：

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY . .
RUN npm install -g pnpm && pnpm install && pnpm build
```

```
FROM node:22-alpine AS runner
WORKDIR /app
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/index.js"]
```

第一阶段负责安装依赖和构建，第二阶段只拿运行需要的文件。

好处：

- 镜像更小
- 不把构建缓存和源码杂物带到运行镜像
- 运行环境更清爽
- 部署更稳定

Next.js 项目常见的 `standalone` 输出，也是为了让最终运行阶段尽量轻。

13. 常用命令速查

镜像：

```
docker images
docker pull 镜像名
docker rmi 镜像名
```

容器：

```
docker ps
docker ps -a
docker stop 容器名
docker start 容器名
docker rm 容器名
docker logs -f 容器名
```

Compose：


```
docker compose up -d
docker compose up -d --build
docker compose ps
docker compose logs -f
docker compose restart
docker compose down
```

清理：

```
docker system df
docker image prune
docker container prune
docker builder prune
docker system prune
```

清理命令要谨慎，尤其是 `docker system prune`，它会删除未使用的镜像、容器、网络 and 缓存。执行前先用 `docker system df` 看一下占用。

14. 常见问题怎么排

端口访问不了：

```
docker compose ps
ss -tunlp | grep 端口
curl http://127.0.0.1:端口
```

容器启动失败：

```
docker compose logs -f 服务名
```

磁盘满了：

```
df -h
docker system df
```

权限不足：

```
ls -ld 目录
id
groups
```

如果是挂载目录权限问题，要检查宿主机目录的 owner 和权限，而不是只盯着容器内部。

Nginx 502：

```
nginx -t
tail -f /var/log/nginx/error.log
curl http://127.0.0.1:后端端口
docker compose logs -f
```

排障思路是从外到内：

浏览器 -> Nginx -> 宿主机端口 -> Docker 端口映射 -> 容器日志 -> 应用进程

15. 我对 Docker 的理解

学 Docker 不要只背命令。命令很容易忘，但关系记住就好：

- Dockerfile 用来制作镜像
- 镜像用来启动容器
- 容器里运行应用
- 端口映射让外部能访问容器
- 挂载让数据离开容器还能保留
- 网络让容器之间能通信
- Compose 把运行规则写成一个文件
- 日志是排错第一入口

如果能把这些关系串起来，Docker 就不再是很玄的东西。它本质上是在帮我们把“部署过程”写成代码。

以前部署靠记忆和手工操作，现在部署靠 Dockerfile 和 Compose 文件。只要文件还在，环境就能被重复创建。这就是 Docker 最有价值的地方。